Unit 4: Model ensembles – motivation, wisdom of crowds, Bagging, Boosting, Random forests, stochastic gradient boosting, heterogeneous ensembles.

Ensemble learning

1. An ensemble model combines predictions from multiple learned models like weak learners (supervised or unsupervised) or base models to produce one final prediction. grouping of reasonably accurate models can outperform an individual model when their errors are not perfectly correlated in terms of accuracy and generalisation.
2. Instead of relying on a single machine learning model which may make errors because of limited training data, noise, overfitting, or the limitations of its learning algorithm ensemble learning aggregates the output of several models KNN, decision tree, linear regression, SVM etc to make final prediction
3. This reduce **bias** (error from assumptions), **variance** (error from sensitivity to data like outliers) and prevent overfitting.

the wisdom of the crowd's principle

means that combining the opinions or decisions of multiple individuals can often produce a better decision than relying on a single individual.

- Different models tend to make different errors, which helps cancel out mistakes when combined. Some models may overfit (memorize data) while others underfit (miss patterns); ensembles balance these issues.
- Ensemble techniques improve performance on new, unseen data, making systems more stable. They help handle data noise and anomalies better than individual models.
- Combining models that excel at different tasks allows for more versatile AI systems.
- Popular AI tools like ChatGPT, DALL·E, and Midjourney use ensemble methods behind the scenes.
- So combining multiple large language models can sometimes match human judgment.
- Hence overall ensemble methods create smarter, more dependable AI by pooling the strengths of many models. This approach transforms a collection of individual predictions into a collective, more accurate decision.

Why use ensembles? Or Motivation

1. Ensemble methods attempt to improve prediction by Combining several models can produce more correct predictions than using only one model.

Example: Three models predict whether an email is spam.

Model 1: Spam, Model 2: Spam, Model 3: Not spam

Using majority voting, the ensemble predicts Spam.

2. **Reducing variance and overfitting in suitable settings** : A model may learn unnecessary details or noise from training data. Combining multiple models can reduce the effect of these fluctuations and help the system perform better on new data.

Example: A single decision tree may classify the training data very accurately but perform poorly on new emails. Combining many decision trees using Random Forest can reduce this problem.

3. **Reducing the impact of errors from individual models :** If one model makes an incorrect prediction, other models may compensate for its mistake. Example: In a medical diagnosis system, three models predict whether a patient has a disease. If one model makes a wrong prediction but the other two are correct, majority voting may give the correct final result.

4. **Improving robustness and generalization.**

- ✓ Robustness: The system is less affected by errors, noise, or small changes in the input.
- ✓ Generalization: The system performs well on data it has not seen during training.

Example: A handwriting recognition system combines several models to recognize digits written in different styles. It may perform better on unfamiliar handwriting than a single model.

5. **Combining different patterns learned by different algorithms.**

Different algorithms can learn different characteristics or relationships from the same dataset. Combining their predictions may capture these patterns more effectively.

For eg: To predict student performance

- ✓ A Decision Tree identifies rule-based patterns.
- ✓ Logistic Regression identifies relationships between features and the probability of passing.
- ✓ A Neural Network learns complex, nonlinear patterns.

Types of Ensemble learning

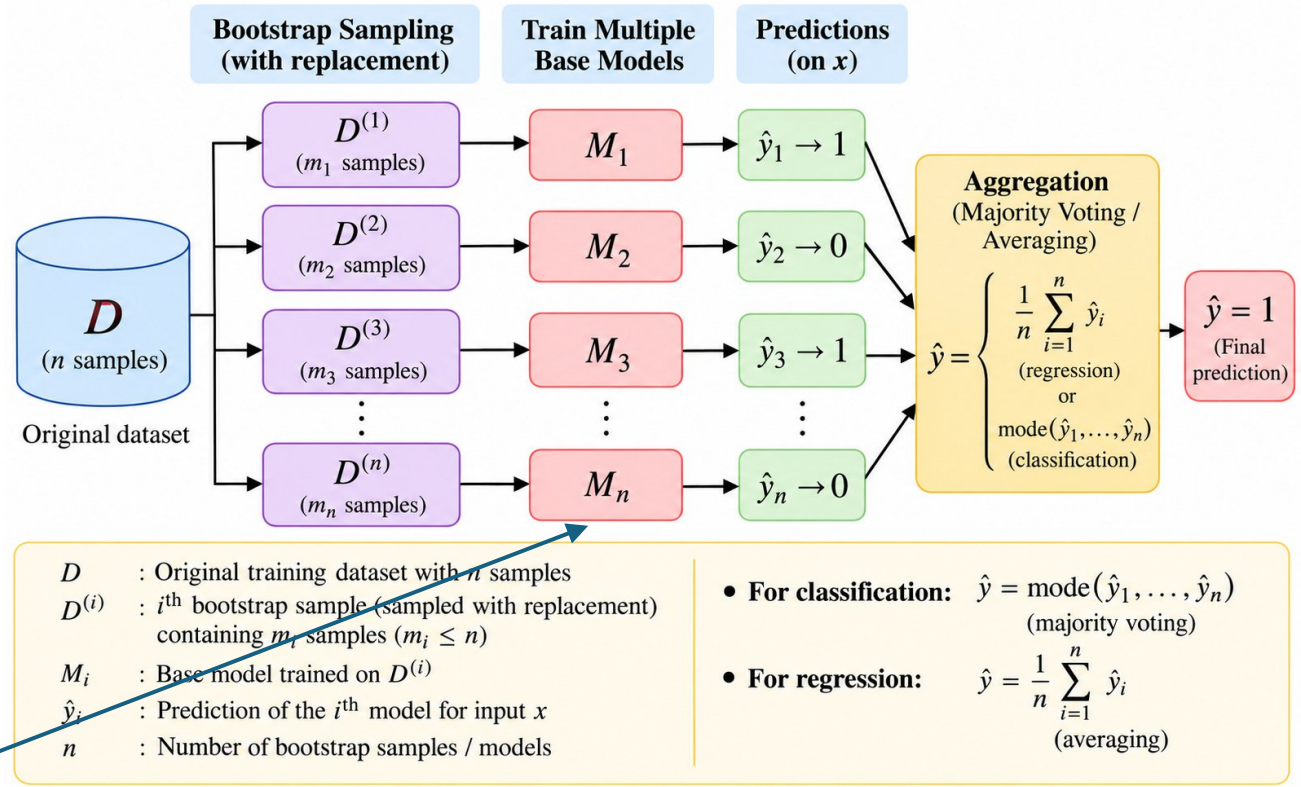
1. **Bagging (Bootstrap Aggregating)**

What it does: Trains multiple models on different random subsets of the training data (sampled with replacement).

How predictions are combined: Usually by averaging (for regression) or majority voting (for classification). Row sampling with replacement.

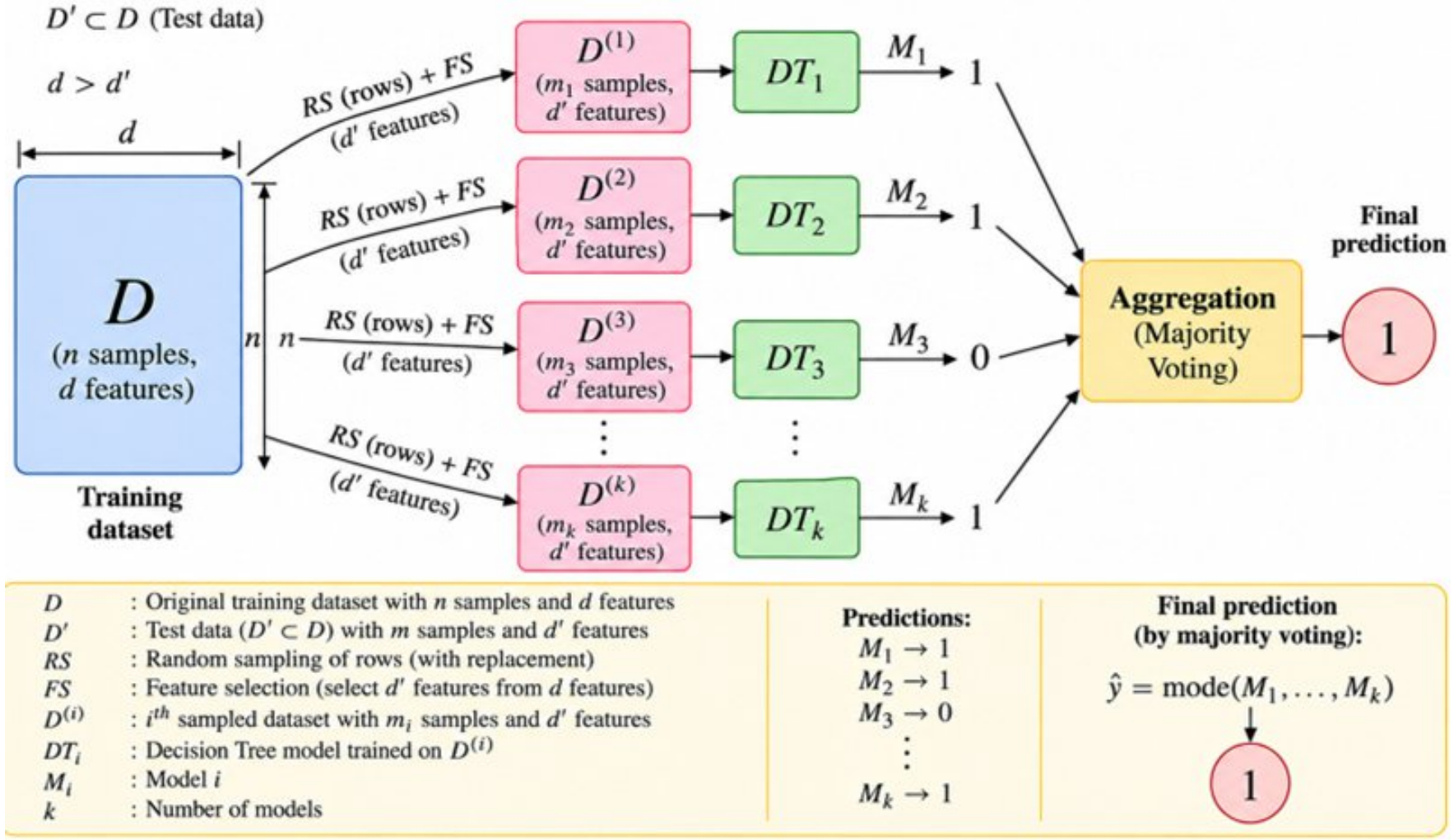
Eg Algo: Random Forest (uses multiple decision trees) used as classifier or regressor

$n > m_1, m_2, .. m_n$



different bootstrap samples generated from dataset D

Random Forest Classifier and Regressor – models M_1 in bagging technique will be replaced by Decision tree and along with row sampling , feature sampling also considered.



Decision Tree has

1. low bias means if tree has complete depth then properly trained for dataset and less error
2. high variance – when new test data comes DT prune to high error so full depth will rise to overfitting. So when majority voting happens high variance change to low variance.

How many Decision tree to be used is the hyper parameter tuning.

If regression, then mean of all output

If we change 200 samples of 1000 samples also output don't affect because of majority voting

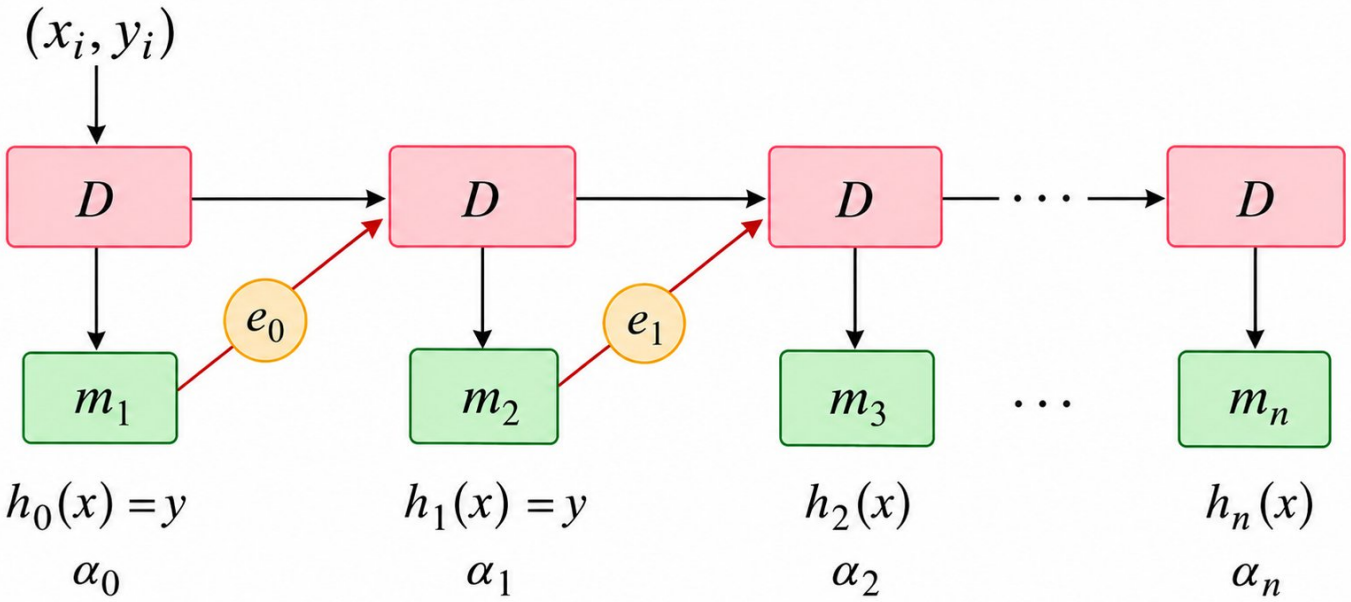
Boosting

- What it does: Trains models sequentially, where each new model focuses on correcting the mistakes of the previous ones.(fine tuning)
- How predictions are combined: Weighted sum of all models.
- Ex Algo: AdaBoost, Gradient Boosting, XGBoost, LightGBM.

- Bagging converts Low Bias, High Variance to Low Bias, Low Variance (Randomisation + Aggregation)
- Boosting is a techniques which converts High Bias, Low Variance model to Low Bias , Low Variance model (Additive Combining)

- Decision tree with high bias , low variance will be less depth with shallow decision tree, high error.

How boosting works



$$f(x) = \alpha_0 h_0(x) + \alpha_1 h_1(x) + \alpha_2 h_2(x) + \cdots + \alpha_n h_n(x)$$

- how to handle error is done using algo like gradient boosting, XGBoost, AdaBoost

- x_i is the input columns, y_i is the output columns
- $m_1, m_2 \ldots m_n$ is the different model which has high bias low variance and apply dataset (D) to them.
- e0,e1..en-1 Is the training error and e0 will be high
- Model ☾ h0(x)=y ☾ y_i-y =e0
- same dataset to m2 + pass e0 the error to h1 (x) model.
- Each subsequent model get trained by previous error and hence reducing error making it Low Bias Low Variance
- this is called boosting by adding error
- how many models need to be tested depend on parameter n_estimator (10 or100 or 1000 etc)
- all intelligence add that will be the final model output f(x)
- alpha is weight added to the model. So depending on less error weight given to that model will be high.

Gradient Boosting for classification and regression

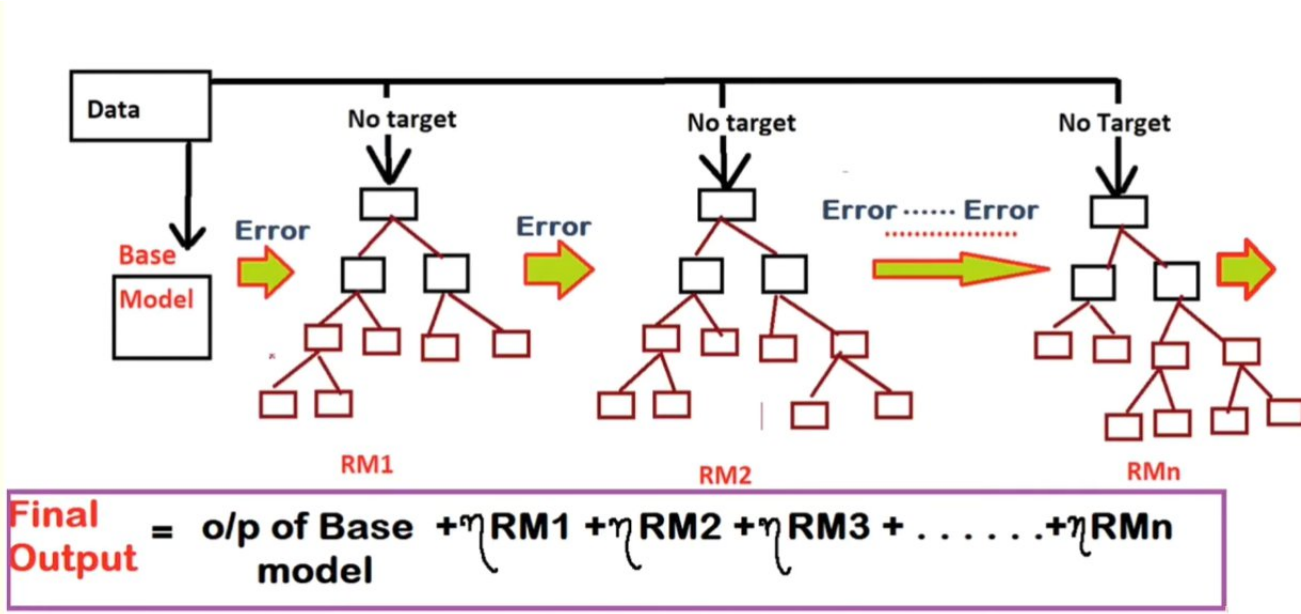
- Gradient Boosting is a machine learning technique used for regression and classification.
- It builds models sequentially, each one correcting the errors of the previous.
- Boosting combines multiple weak learners (like small decision trees) to create a strong learner.
- Each model focuses on the errors made by previous models.

1. Learning happens by optimizing the loss function.(l1 or l2 loss or mse, rmse etc) –minimising the loss

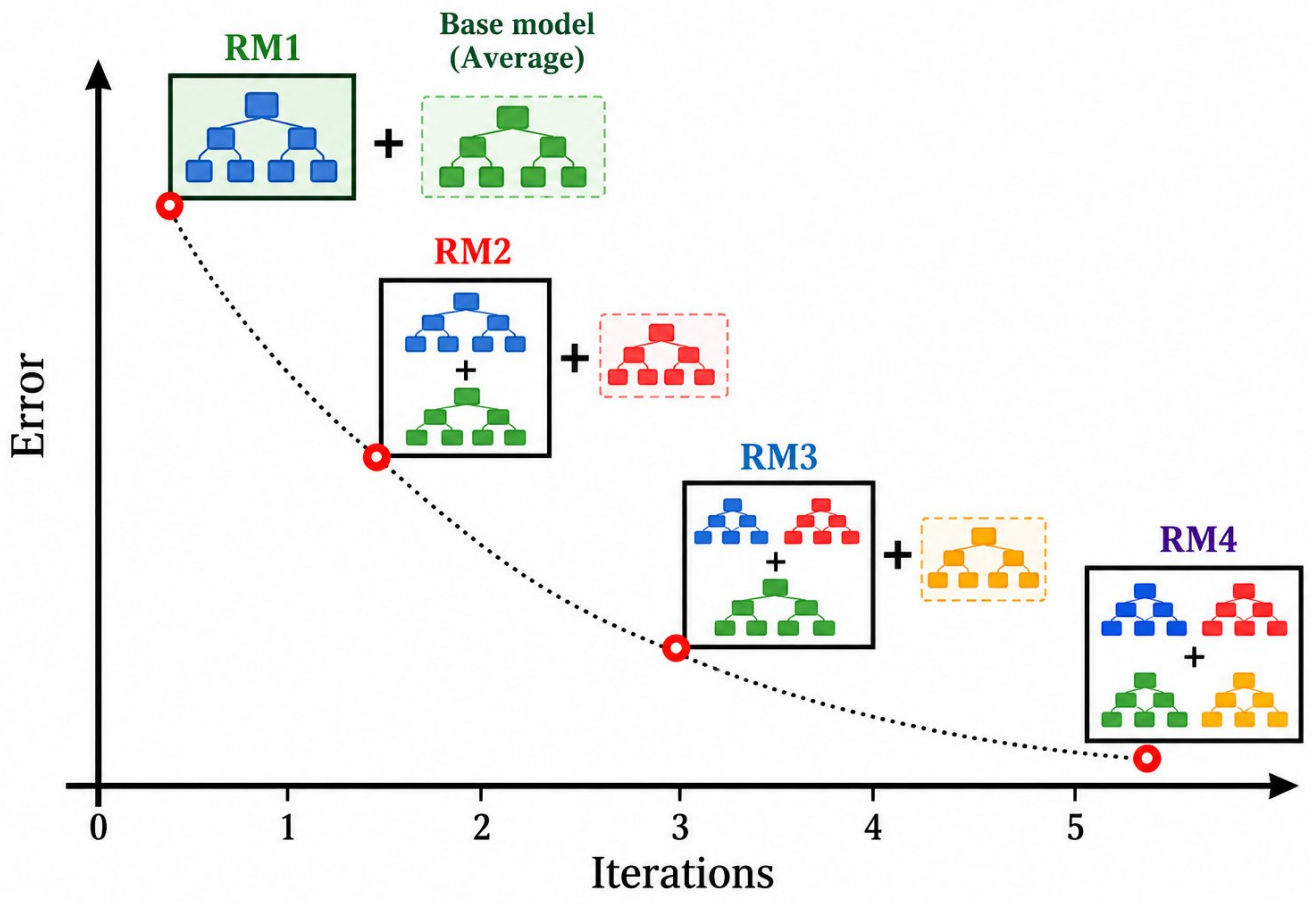
2. Uses two types of base estimators:

First is an average-type model. (will have learning rate

Second is a decision tree with full depth.



Error with iterations



step	Action	Explanation
1	Start with an initial prediction	Usually mean (regression) or log odds (classification).
2	Compute residuals	Errors between actual and predicted values.
3	Fit a new tree on residuals	New model learns where the first model went wrong.
4	Update predictions	Add the new model’s predictions (scaled by learning rate).
5	Repeat	Keep improving until stopping criteria are met.

new model = base model +learning rate * residue model

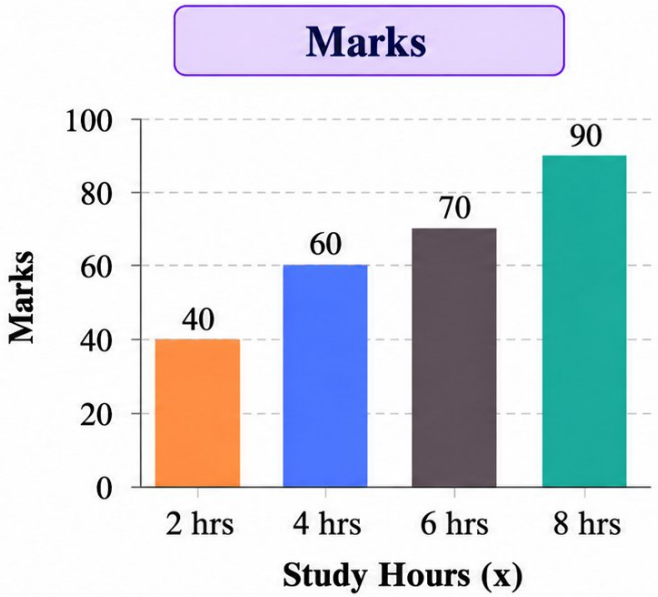
Numerical example of Gradient Boosting using two models or two decision tree

Each tree corrects previous mistakes:

$$F(x) = F_0(x) + \eta \Sigma h_m(x)$$

η (learning rate) controls how much correction is applied.

Student	Study Hours (x)	Marks (y)
A	2	40
B	4	60
C	6	70
D	8	90



Step 1 – Initial Prediction

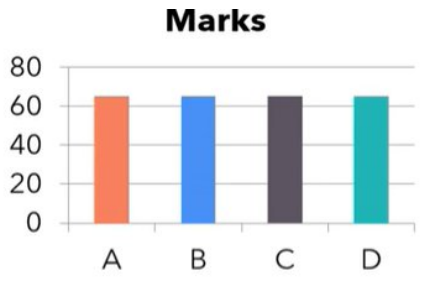
Assume average marks = 65
So, the first model predicts **65 for all** students.

(40+60+70+90)/4

Student	Study Hours (x)	Marks (y)	First Prediction	Residual
A	2	40	65	-25
B	4	60	65	-5
C	6	70	65	5
D	8	90	65	25

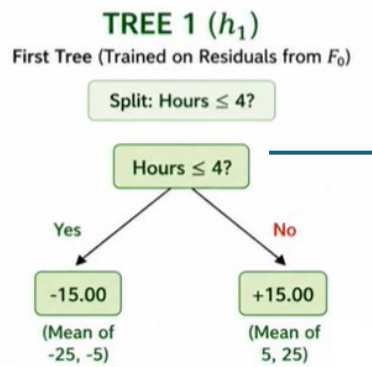
For this split:
Left node ($x \leq 4$): average residual = (-25 + -5) / 2 = **-15**
Right node ($x > 4$): average residual = (+5 + +25) / 2 = **+15**
So, the **tree prediction rule** is:

$$h_1(x) = \begin{cases} -15, & x \leq 4 \\ +15, & x > 4 \end{cases}$$



Student	Study Hours (x)	Marks (y)	First Prediction	Residual
A	2	40	65	-15
B	4	60	65	-15
C	6	70	65	15
D	8	90	65	15

h1(x) output



Take values so that Tree should be balanced

Step 2 – Train Model on Residuals

Train a small tree on the residuals:

It learns that more study hours → positive residual

Less study hours → negative residual

So, the tree outputs approximate correction values.

Step 3-Update the Model

Now apply learning rate η (say 0.1):

$$F_1(x) = F_0(x) + \eta h_1(x)$$

Student	Old Pred (F_0)	$h_1(x)$	η=0.1	New Pred
A	65	-15	0.1	63.5
B	65	-15	0.1	63.5
C	65	+15	0.1	66.5
D	65	+15	0.1	66.5

y	Student	Old Pred (F_0)	$h_1(x)$	New Pred	Residual
40	A	65	-15	63.5	-23.5
60	B	65	-15	63.5	-3.5
70	C	65	+15	66.5	3.5
90	D	65	+15	66.5	23.5

Y-new pred

- Recalculate residuals and train new trees.
- Continue until residuals ≈ 0.
- Final model = sum of all small trees.

$$F2(x) = F1(x) + \eta \Sigma h_m(x)$$

$$Residual = y - F1(x)$$

F1(x)

Student	Study Hours (x)	Marks (y)	New Pred	Residual	h2(x)	η=0.1	New Pred
A	2	40	63.5	-23.5	-13.5	0.1	62.15
B	4	60	63.5	-3.5	-13.5	0.1	62.15
C	6	70	66.5	3.5	13.5	0.1	67.85
D	8	90	66.5	23.5	13.5	0.1	67.85

F2(x)

Student	Study Hours (x)	Marks (y)	New Pred	Residual	h3(x)	η=0.1	New Pred
A	2	40	62.15	-22.15	-12.15	0.1	60.935
B	4	60	62.15	-2.15	-12.15	0.1	60.935
C	6	70	67.85	2.15	12.15	0.1	69.065
D	8	90	67.85	22.15	12.15	0.1	69.065

Final Model in Gradient Boosting

After multiple iterations, the model becomes a **sum of many small trees**:

$$F(x) = F_0(x) + \eta h_1(x) + \eta h_2(x) + \eta h_3(x) + \cdots + \eta h_M(x)$$

If $\eta = 1$, model corrects too much, swinging predictions too far.
If $\eta = 0.1$, model corrects **gently** and can refine over multiple trees.

Student	Residual (h_1(x))	$\eta = 1$	$\eta = 0.1$
A	-15	65 - 15 = 50	65 - 1.5 = 63.5
D	+15	65 + 15 = 80	65 + 1.5 = 66.5

η Value	Behavior	Notes
1.0	Very fast learning, high risk of overfitting	Rarely used
0.3	Moderate speed	Default in XGBoost
0.1	Balanced — commonly used	Works well with 100–300 trees
0.01	Very slow learning, needs more trees	High accuracy but long training time

Gradient Boosting: Mathematical Idea

Training Data

Student	Hours (x)	Actual (y)
A	2	40
B	4	60
C	6	70
D	8	90

Learning rate (η) = 0.1

Initial Model (F_0)

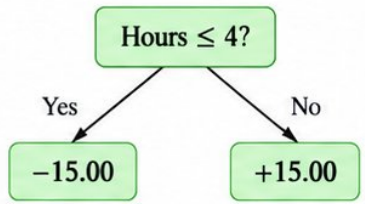
Mean of $y = (40+60+70+90) / 4 = 65$

$F_0(x) = 65$ (for all x)

Student	$F_0(x)$	Residuals $(y - F_0(x))$
A	65.00	−25.00
B	65.00	−5.00
C	65.00	5.00
D	65.00	25.00

Tree 1 (h_1)

Split: $x \leq 4$



$h_1(x) = [-15.00, -15.00, 15.00, 15.00]$

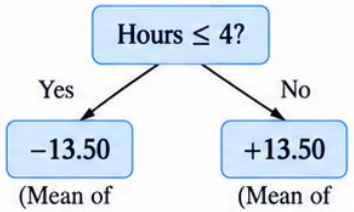
Student	$F_0(x)$	$\eta \cdot h_1(x)$	$F_1(x) = F_0(x)+\eta h_1(x)$	Residuals $(y - F_1(x))$
A	65.00	−1.50	63.50	−23.50
B	65.00	−1.50	63.50	−3.50
C	65.00	1.50	66.50	3.50
D	65.00	1.50	66.50	23.50

$F_1(x) = [63.50, 63.50, 66.50, 66.50]$

Tree 2 (h_2)

Using residuals from $F_1(x)$

Split: $x \leq 4$



$h_2(x) = [-13.50, -13.50, 13.50, 13.50]$

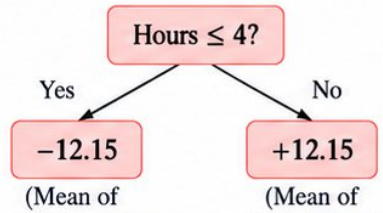
Student	$F_1(x)$	$\eta \cdot h_2(x)$	$F_2(x) = F_1(x)+\eta h_2(x)$	Residuals $(y - F_2(x))$
A	63.50	−1.35	62.15	−22.15
B	63.50	−1.35	62.15	−2.15
C	66.50	1.35	67.85	2.15
D	66.50	1.35	67.85	22.15

$F_2(x) = [62.15, 62.15, 67.85, 67.85]$

Tree 3 (h_3)

Using residuals from $F_2(x)$

Split: $x \leq 4$



$h_3(x) = [-12.15, -12.15, 12.15, 12.15]$

Student	$F_2(x)$	$\eta \cdot h_3(x)$	$F_3(x) = F_2(x)+\eta h_3(x)$	Residuals $(y - F_3(x))$
A	62.15	−1.215	60.935	−20.935
B	62.15	−1.215	60.935	−0.935
C	67.85	1.215	69.065	0.935
D	67.85	1.215	69.065	20.935

$F_3(x) = [60.935, 60.935, 69.065, 69.065]$

Another example

Step 1: Original training data

Suppose we have four houses.

House	Area (sq. ft.)	Actual price y (₹ lakhs)
A	1,000	100
B	1,500	120
C	2,000	140
D	2,500	160

Our objective is to predict the house prices using the area.

Step 2: Build the base model

For squared-error regression, the initial prediction is the mean of the actual prices.

$$F_0(x) = \frac{100 + 120 + 140 + 160}{4} = 130$$

Therefore, the base model predicts ₹130 lakhs for every house.

House	Actual price	Base prediction	Residual $y - F_0(x)$
A	100	130	-30
B	120	130	-10
C	140	130	+10
D	160	130	+30

A negative residual means the model overpredicts the price; a positive residual means it underpredicts the price.

Step 3: Train the first model (RM1)

Now, train a regression tree using:

- **Independent variable:** House area
- **Target variable:** Residual from Step 2

For a simple example, let RM1 split the houses into two groups:

- Area up to 1,500 sq. ft.: average residual $= (-30 - 10)/2 = -20$
- Area above 1,500 sq. ft.: average residual $= (10 + 30)/2 = +20$

Thus, RM1 predicts the average residual within each group.

Thus, RM1 predicts the average residual within each group.

House	Area	Residual target	RM1 predicted residual
A	1,000	-30	-20
B	1,500	-10	-20
C	2,000	+10	+20
D	2,500	+30	+20

Step 4: Update predictions using RM1

The updated prediction is:

$$F_1(x) = F_0(x) + \eta\, RM1(x)$$

Since $\eta = 1$,

$$F_1(x) = 130 + RM1(x)$$

F0(x)

F1(x)

Actual price

House	Base prediction	RM1 output	Updated prediction	New residual
A	130	-20	110	$100 - 110 = -10$
B	130	-20	110	$120 - 110 = +10$
C	130	+20	150	$140 - 150 = -10$
D	130	+20	150	$160 - 150 = +10$

Notice that RM1 has improved the predictions for houses A and D, but errors remain for all four houses.

Step 5: Train the second model (RM2)

RM2 is trained using the **new residuals** as its target.

House	Area	New residual (target for RM2)
A	1,000	-10
B	1,500	+10
C	2,000	-10
D	2,500	+10

For illustration, suppose RM2 is a sufficiently flexible regression tree that learns these four residual values exactly.

House	RM1-updated prediction	RM2 predicted residual
A	110	-10
B	110	+10
C	150	-10
D	150	+10

This is same previous residual because applying decision average will become 0 so took previous RM1

Step 6: Update predictions using RM2

$$F_2(x) = F_1(x) + \eta\, RM2(x)$$

With $\eta = 1$:

House	Actual price	Prediction after RM1	RM2 output	Final prediction after RM2
A	100	110	-10	100
B	120	110	+10	120
C	140	150	-10	140
D	160	150	+10	160

AdaBoost

1. Learning happens by weight updates.

    Increase weights for misclassified instances.

    Decrease weights for correctly classified instances.

2. Uses stumps for every estimator.

Depth = 1.

Steps for AdaBoost Algorithm

Step 1: Initialize the weights as 1/n for every one of the n observations.

Step 2: Select the best feature according to the lowest Gini impurity or highest information gain, and calculate the total error.

Step 3: Calculate the performance of the decision stump.

Step 4: Calculate the new weights for each observation:

Increase the weight for a misclassified observation.

Decrease the weight for a correctly classified observation.

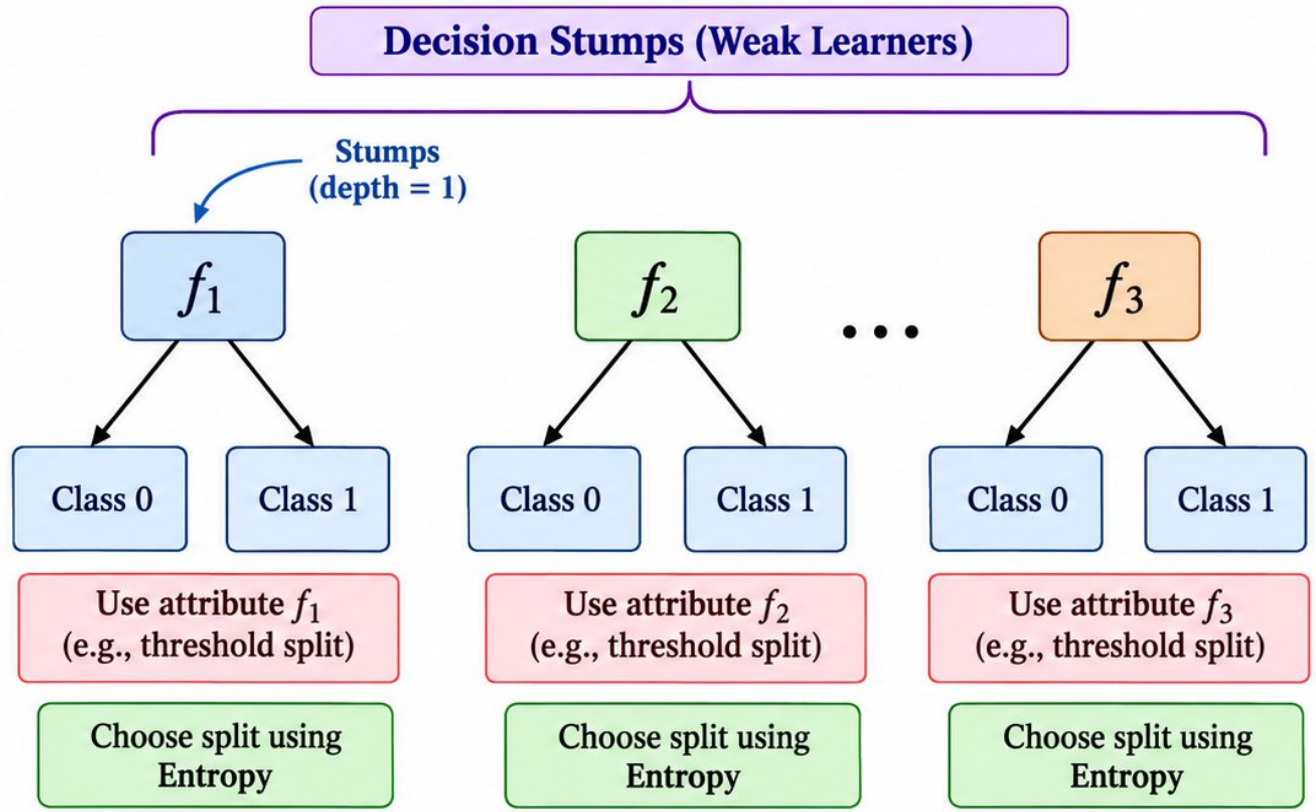
Step 5: Normalize the new weights so that the sum of all weights is 1.

Step 6: Repeat from Step 2 until the configured number of estimators is reached or the desired performance is achieved.

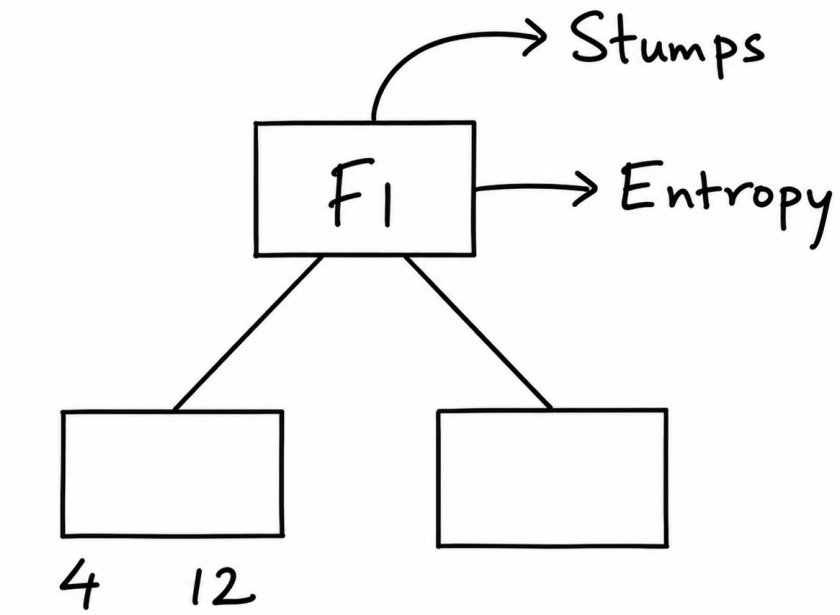
AdaBoost (Boosting Technique)

$$w = \frac{1}{n} = \frac{1}{7}$$

f_1	f_2	f_3	O/P	Sample Weight
x_{11}	x_{12}	x_{13}	y_1	1/7
x_{21}	x_{22}	x_{23}	y_2	1/7
x_{31}	x_{32}	x_{33}	y_3	1/7
x_{41}	x_{42}	x_{43}	y_4	1/7
x_{51}	x_{52}	x_{53}	y_5	1/7
x_{61}	x_{62}	x_{63}	y_6	1/7
x_{71}	x_{72}	x_{73}	y_7	1/7



Select feature with less entropy



Total Error = $\frac{1}{7}$

AdaBoost (Boosting Technique)

$$w = \frac{1}{n} = \frac{1}{7}$$

	f_1	f_2	f_3	O/P	Sample Weight
1	Yes	Yes	Yes	$y_1 = Yes$	1/7
2	Yes	No	No	$y_2 = No$	1/7
3	No	Yes	No	$y_3 = Yes$	1/7
4	No	No	Yes	$y_4 = No$	1/7
5	Yes	Yes	No	$y_5 = Yes$	1/7
6	No	No	Yes	$y_6 = No$	1/7
7	No	No	No	$y_7 = No$	1/7

Total Error = $\frac{1}{7}$

Step 3: Performance of the Stump

$$\alpha = \frac{1}{2}\log_e\left(\frac{1 - TE}{TE}\right)$$

TE = Total Error = $\frac{1}{7}$ (from the table)

Performance of Stump:

$$\alpha = \frac{1}{2}\log_e\left[\frac{1 - \frac{1}{7}}{\frac{1}{7}}\right]$$

$$= \frac{1}{2}\log_e\left[\frac{\frac{6}{7}}{\frac{1}{7}}\right]$$

$$= \frac{1}{2}\log_e[6]$$

$$= 0.896$$

Weight (α) of this stump = 0.896

- Update weights of incorrectly and correctly classified sample

In Correctly classified

Updated weight

Performance (α) = 0.895

$$\text{New sample weight} = \text{Old weight} \times e^{Performance}$$

$$= \frac{1}{7} \times e^{0.895}$$

$$\approx 0.349$$

New sample weight ≈ 0.349

Correctly classified

Updated Sample Weight

$$\text{New Sample Weight} = \text{Old Weight} \times e^{-Performance}$$

$$= \frac{1}{7} \times e^{-0.895}$$

$$\approx 0.058$$

AdaBoost

Dataset :-

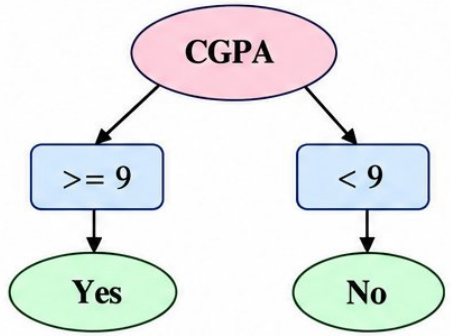
CGPA (1)	Interactiveness (2)	Practical Knowledge (3)	Communication Skill (4)	Job Profile (5)	
>= 9	Yes	Good	Good	Yes	1
< 9	No	Good	Moderate	Yes	2
>= 9	No	Average	Moderate	No	3
< 9	No	Average	Good	No	4
>= 9	Yes	Good	Moderate	Yes	5
>= 9	Yes	Good	Moderate	Yes	6

N = 6

Initial weight for each sample:

$$w_i = \frac{1}{N} = \frac{1}{6}$$

Decision Stump for CGPA :-

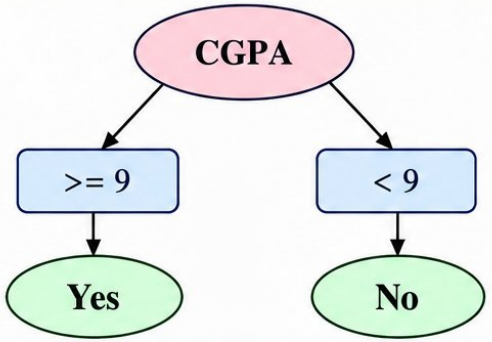


CGPA	Prediction ($h_1(x)$)	Weight
>= 9	Yes	1/6
< 9	No	1/6
>= 9	Yes	1/6
< 9	No	1/6
>= 9	Yes	1/6
>= 9	Yes	1/6

Wrongly classified compare to actual in col 5 in above table

Selected stump with on low entropy or low Gini index or high information gain

Decision Stump for CGPA



Student	CGPA	Prediction ($h_1(x)$)	Actual (Job Profile)	Correct?	Weight (w_i)
1	>= 9	Yes	Yes	✓	1/6
2	< 9	No	Yes	✗	1/6
3	>= 9	Yes	No	✗	1/6
4	< 9	No	No	✓	1/6
5	>= 9	Yes	Yes	✓	1/6
6	>= 9	Yes	Yes	✓	1/6

2 wrong detected

$$E_x = E_{CGPA}$$

$$= 2 \times \frac{1}{6}$$

$$E_{CGPA} = \frac{2}{6}$$

Weighted error of this stump:

$$E_{CGPA} = 2 \times \frac{1}{6}$$

$$= \frac{2}{6}$$

Total error

Step 3: Performance of the Stump

$$\alpha = \frac{1}{2}\log_e\left(\frac{1-TE}{TE}\right)$$

TE = Total Error = $\frac{1}{3}$ (from the table)

Performance of Stump:

$$\alpha = \frac{1}{2}\log_e\left[\frac{1-\frac{1}{3}}{\frac{1}{3}}\right]$$

$$= \frac{1}{2}\log_e\left[\frac{\frac{2}{3}}{\frac{1}{3}}\right]$$

$$= \frac{1}{2}\log_e[2]$$

$$= 0.347$$

Weight (α) of this stump = 0.347

Update weights

- If the prediction is wrong (misclassified) → increase weight
- If the prediction is correct (classified correctly) → decrease weight

$$w_{new} = w_{old} \times e^{\alpha} \quad \text{(if incorrect)}$$

$$w_{new} = w_{old} \times e^{-\alpha} \quad \text{(if correct)}$$

S.No	Initial wts		wts (after update)	Normalized wts
1	1/6		0.11	0.12
2	1/6	✓	0.23	0.25
3	1/6	✓	0.23	0.25
4	1/6		0.11	0.12
5	1/6		0.11	0.12
6	1/6		0.11	0.12

$$\alpha = 0.347$$

$$e^{\alpha} = e^{0.347} \approx 1.414$$

$$e^{-\alpha} = e^{-0.347} \approx 0.707$$

Normalize weights (Z)

$$Z = \frac{w_t'}{\text{Total}}$$

Total = $4 \times 0.11 + 2 \times 0.23$
= $0.44 + 0.46$
= 0.9

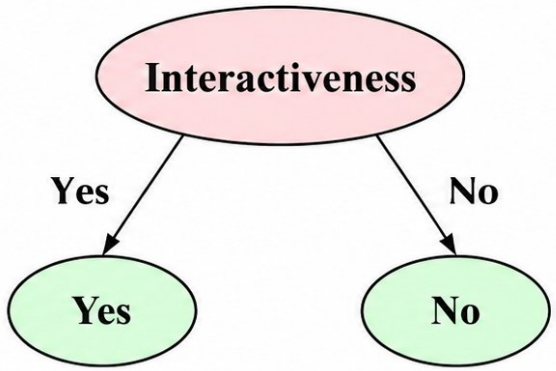
Normalized weights:

$\frac{0.11}{0.9} = 0.12$, $\frac{0.23}{0.9} = 0.25$

Repeat the same steps for all attributes

Interactiveness

Interactiveness	Predicted Job Offer	Actual Job Offer	Weight
Yes	Yes	Yes	0.12
No	No	Yes	0.25
No	No	No	0.25
No	No	No	0.12
Yes	Yes	Yes	0.12
Yes	Yes	Yes	0.12



$$E_{interactiveness} = \frac{1}{6}$$

Error = 1* 0.25=0.25

AdaBoost – Weight Update Calculation (Example)

1. Stump weight (for Interactiveness)

$$\alpha = \frac{1}{2}\ln\left(\frac{1-\varepsilon}{\varepsilon}\right) \quad \Rightarrow \quad \alpha = \frac{1}{2}\ln\left(\frac{1-\varepsilon}{\varepsilon}\right) = 0.549$$

$$\alpha_{int} = \mathbf{0.549}$$
(Interactiveness stump)

2. Update rule for weights

If prediction is correct:

$$w_{new} = w_{old} \times e^{-0.549}$$

If prediction is incorrect:

$$w_{new} = w_{old} \times e^{0.549}$$

Exponential values

$$e^{-0.549} \approx 0.577$$

$$e^{0.549} \approx 1.732$$

S.No	Old weight (w_{old})	Prediction (Actual vs Predicted)	w_{new} (unnormalized)	Normalized weight (w_{new}/Z)
1	0.12	Correct	$0.12 \times 0.577 = 0.069$	$0.069 / 0.783 \approx 0.088$
2	0.25	Incorrect	$0.25 \times 1.732 = 0.433$	$0.433 / 0.783 \approx 0.553$
3	0.25	Correct	$0.25 \times 0.577 = 0.144$	$0.144 / 0.783 \approx 0.184$
4	0.12	Correct	$0.12 \times 0.577 = 0.069$	$0.069 / 0.783 \approx 0.088$
5	0.12	Correct	$0.12 \times 0.577 = 0.069$	$0.069 / 0.783 \approx 0.088$
6	0.12	Correct	$0.12 \times 0.577 = 0.069$	$0.069 / 0.783 \approx 0.088$

3. Total unnormalized weight (Z)

$$Z = 4 \times 0.069 + 1 \times 0.433 + 1 \times 0.144$$
$$= 0.276 + 0.433 + 0.144$$
$$= \mathbf{0.783}$$

$$\boldsymbol{Z = 0.783}$$

4. Normalized weights

Correct: $0.069 / 0.783 \approx 0.088$

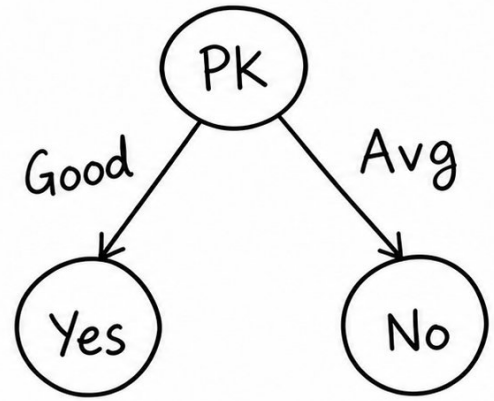
Correct (row 3): $0.144 / 0.783 \approx 0.184$

Incorrect: $0.433 / 0.783 \approx 0.553$

Sum of normalized weights: $4 \times 0.088 + 0.553 + 0.184 = 0.352 + 0.553 + 0.184 = 1.0$

3) Practical Knowledge

	PK	Pred	Act	Wt
1	Good	Yes	Yes	0.08
2	Good	Yes	Yes	0.50
3	Avg	No	No	0.184
4	Avg	No	No	0.08
5	Good	Yes	Yes	0.08
6	Good	Yes	Yes	0.08



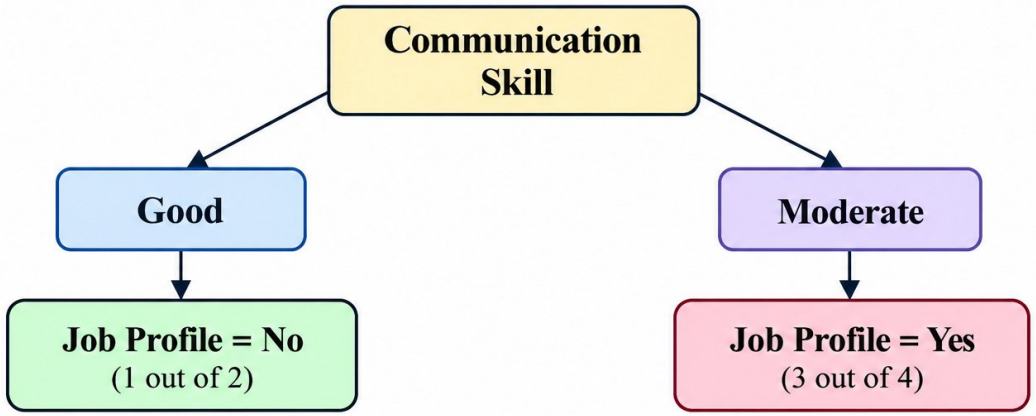
Error = 0

So no change in weights

Communication skill

Dataset

CGPA (1)	Interactiveness (2)	Practical Knowledge (3)	Communication Skill (4)	Job Profile (5)	
>= 9	Yes	Good	Good	Yes	1
< 9	No	Good	Moderate	Yes	2
>= 9	No	Average	Moderate	No	3
< 9	No	Average	Good	No	4
>= 9	Yes	Good	Moderate	Yes	5
>= 9	Yes	Good	Moderate	Yes	6



Final Predicted Value using Weighted Decision Stumps

S.No	d_1 ($\alpha_1 = 0.347$)	d_2 ($\alpha_2 = 0.549$)	d_3 ($\alpha_3 = -0.548$)	Wt (α_{avg})	Final Predicted
1	Yes	Yes	Yes	0.348	**Yes**
2	No	No	No	−0.348	**No**
3	Yes	No	No	0.346	**Yes**
4	No	No	Yes	−1.444	**No**
5	Yes	Yes	No	1.444	**Yes**
6	Yes	Yes	No	1.444	**Yes**

Final prediction formula

$$F(x) = \sum_{i=1}^{3} \alpha_i h_i(x)$$

where,

- $h_i(x) = 1$ if prediction is Yes
- $h_i(x) = -1$ if prediction is No
- $\alpha_1 = 0.347,\ \alpha_2 = 0.549,\ \alpha_3 = -0.548$

$$F(x) = 0.347(1) + 0.549(1) + (-0.548)(1)$$
$$= 0.347 + 0.549 - 0.548 = \mathbf{0.347}$$

1 Initialize weights

Assign equal weights to all training samples.

$$w_i = \frac{1}{n} \quad \text{for } i = 1, 2, \ldots, n$$

n = number of training samples

2 Check predictions for each attribute (weak learner)

For each attribute (stump), find how many samples are classified correctly and incorrectly.

Attribute (stump)	Correct	Incorrect
d_1	c_1	m_1
d_2	c_2	m_2
...	...	...
d_T	c_T	m_T

3 Find weighted error for each attribute

$$\varepsilon_j = \sum_{i=1}^{n} w_i \, I\left(h_j(x_i) \neq y_i\right)$$

where, $I = 1$ if prediction is incorrect
$I = 0$ if prediction is correct

4 Compute stump performance (weight of each attribute)

$$\alpha_j = \frac{1}{2} \log_e \left(\frac{1 - \varepsilon_j}{\varepsilon_j} \right)$$

Compute α_j for each attribute using its weighted error ε_j

5 Update sample weights

$$w_i^{new} = w_i^{old} \times \begin{cases} e^{\alpha_j} & \text{if prediction is incorrect} \\ e^{-\alpha_j} & \text{if prediction is correct} \end{cases}$$

Incorrectly classified samples get higher weight.
Correctly classified samples get lower weight.

6 Normalize weights

$$w_i = \frac{w_i^{new}}{\sum_{i=1}^{n} w_i^{new}}$$

Normalize the updated weights so that the sum of weights is 1.

7 Repeat

Repeat steps 2–6 for all attributes (weak learners) or for a fixed number of iterations T.

8 Final predicted value

$$P = \sum_{j=1}^{T} \alpha_j h_j(x)$$

$h_j(x) = +1$ if prediction is Yes
$h_j(x) = -1$ if prediction is No

If $P > 0$ ⇒ **Predict Yes**
If $P < 0$ ⇒ **Predict No**

XGBoost

- XGBoost stands for eXtreme Gradient Boosting — is an optimized and efficient implementation of the gradient boosting algorithm, designed to deliver high predictive accuracy on structured or tabular datasets.
- It is widely used because of its speed, computational efficiency, and strong predictive performance. It is applied in various domains, including finance, healthcare, and e-commerce.
- **XGBoost creates decision trees one after another, and each new tree tries to improve the predictions made by the trees already built.**
- Although an individual decision tree (weak learners) may be relatively simple, combining multiple trees sequentially enables XGBoost to produce powerful and accurate predictions.

Why XGBoost is Fast & Efficient

1. Parallelized Split-Finding: Candidate splits across features are evaluated in parallel, not one at a time.

Example: To predict whether a student passes, the model considers:

Study hours: 2, 4, 6, 8 Attendance: 50%, 60%, 80%, 90%

XGBoost can evaluate splits on study hours and attendance simultaneously using parallel processing, helping speed up training.

2. Cache-Aware Access

Data is stored in a compressed column block layout, so repeated scans stay CPU-cache friendly.

Example: Suppose a dataset contains marks, attendance, and study hours for 10,000 students. XGBoost stores feature data in an organized format so the processor can access values efficiently while evaluating tree splits.

3. Sparsity-Aware Splits : Missing values learn a default branch direction automatically — no manual imputation needed.

Student	Study hours	Result
A	2	Fail
B	6	Pass
C	Missing	?
D	8	Pass

During training, XGBoost learns whether missing study-hour values should follow the left or right branch at a split.

4. Weighted Quantile Sketch

For huge datasets, split candidates are approximated from weighted percentiles instead of every value.

example: Suppose a dataset contains one million students' marks. Instead of testing every distinct mark as a split point, XGBoost selects representative candidates, such as 40, 60, and 80.

5. Built-in Regularization : Helps the model generalize better to unseen data.

L1 and L2 penalties are part of the objective itself, not a separate post-processing step.

Example: A tree with 20 leaves may fit the training data very closely, including noise. Regularization can penalize this complexity and encourage a simpler tree with fewer leaves.

- L1 regularization: Can encourage some leaf weights to become zero.
- L2 regularization: Penalizes large leaf weights.

6. Shrinkage (Learning Rate)

The learning rate controls how much each new tree contributes to the ensemble's prediction. Helps control how quickly the model learns and can improve generalization.

How XGBoost Works: Every boosting round repeats the same cycle.

1. Base Learners

Start with simple decision trees.

2. Second-Order Boosting

Use gradient + Hessian to see how error behaves.

3. Best Split (Gain)

Score candidate splits and pick the strongest one.

4. Regularization

Penalize complex trees to avoid overfitting.

5. Final Prediction

Sum every tree's small contribution.

1. Decision Trees as Base Learners

XGBoost builds its model using decision trees as base learners. But instead of relying on just one tree, it builds many trees, one after another and each new tree focuses on correcting the mistakes made by the trees built before it.

no single tree needs to be perfect, because the group corrects itself over time.

How the trees work

Tree 1 — fits the residual error.

Tree 2 — fits the remaining prediction error.

Tree 3 — fits the error that remains after the earlier trees.

2. Second-Order Boosting

The algorithm minimizes the loss function using calculus. But XGBoost goes a step further than classic gradient boosting: **it uses both the gradient (first derivative) and the Hessian (second derivative) of the loss to decide how to update predictions** , a Newton-style step, not pla

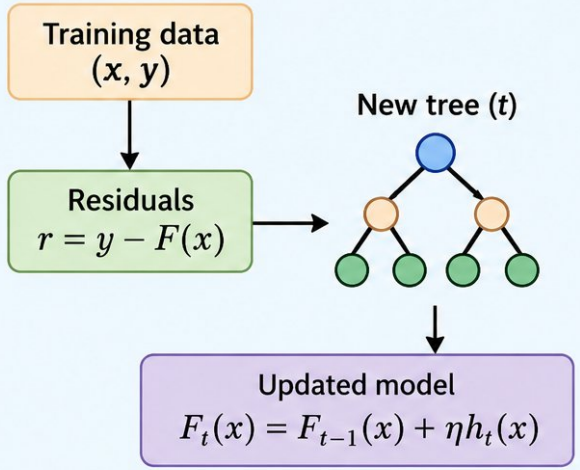
Newton-style optimization is a method that uses both the first and second derivatives of a loss function to determine how to update a prediction or parameter.
it considers not only which direction to move but also the curvature of the loss function to decide how large the update should be.

Classic Gradient Boosting

- Fits each new tree to the simple residual:

$$r = y - F(x)$$

- Only uses first-order information about the error.



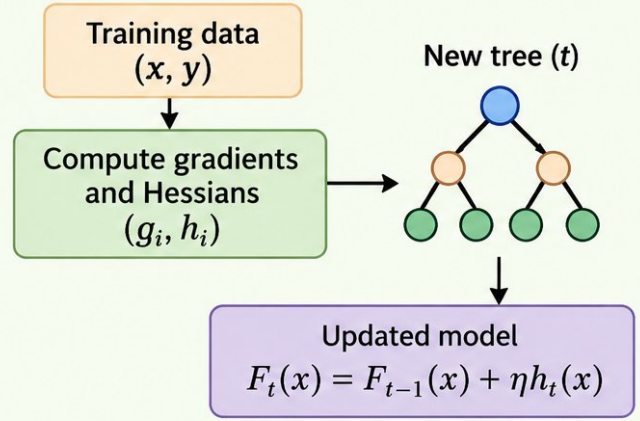
XGBoost

- Computes how the error behaves, using:

$$g_i = \frac{\partial L}{\partial \hat{y}_i} \quad \text{(Gradient)}$$

$$h_i = \frac{\partial^2 L}{\partial \hat{y}_i^2} \quad \text{(Hessian)}$$

- Second-order information can enable faster, more precise optimization.



3 Finding the Best Split (Gain)

For every node, XGBoost tests candidate split points and scores each with a **Gain** formula — how much the split would reduce loss versus keeping the node whole. The split with the highest gain wins.

$$\text{Gain} = \tfrac{1}{2}\left[\frac{G_L^2}{H_L+\lambda} + \frac{G_R^2}{H_R+\lambda} - \frac{G^2}{H+\lambda}\right]$$

G_L, H_L
Sum of gradients and Hessians in the left child.

G_R, H_R
Sum of gradients and Hessians in the right child.

G, H
Sum of gradients and Hessians in the parent node.

λ
Regularization strength (L2 penalty).

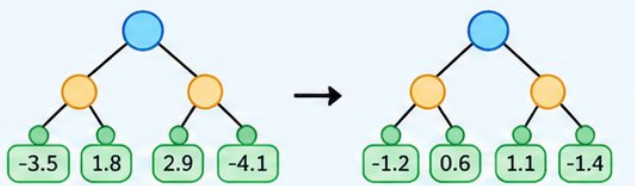
4 Regularization in Tree Building

While building each tree, XGBoost applies **regularization terms** directly inside the split and leaf-weight calculations. This keeps trees from growing overly complex and helps control overfitting.

L2 (λ)

Shrinks leaf weights toward zero — smoother, less extreme predictions.

$$\Omega(w) = \frac{1}{2}\lambda \sum_j w_j^2$$



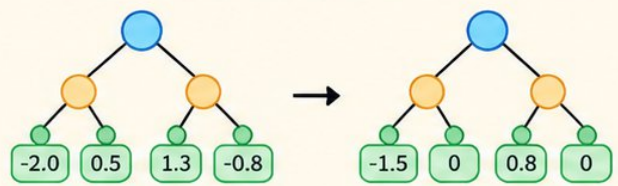
Without L2 (large weights)

With L2 (shrunk weights)

L1 (α)

Encourages sparsity — can push some leaf weights to exactly zero.

$$\Omega(w) = \alpha \sum_j |w_j|$$



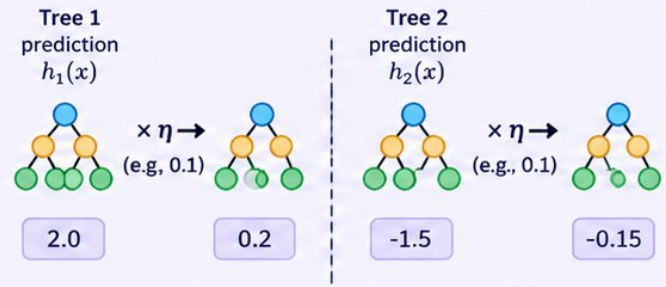
Without L1 (all leaves non-zero)

With L1 (some leaves zero)

Shrinkage (η)

Scales down every tree's contribution, trading more trees for better generalization.

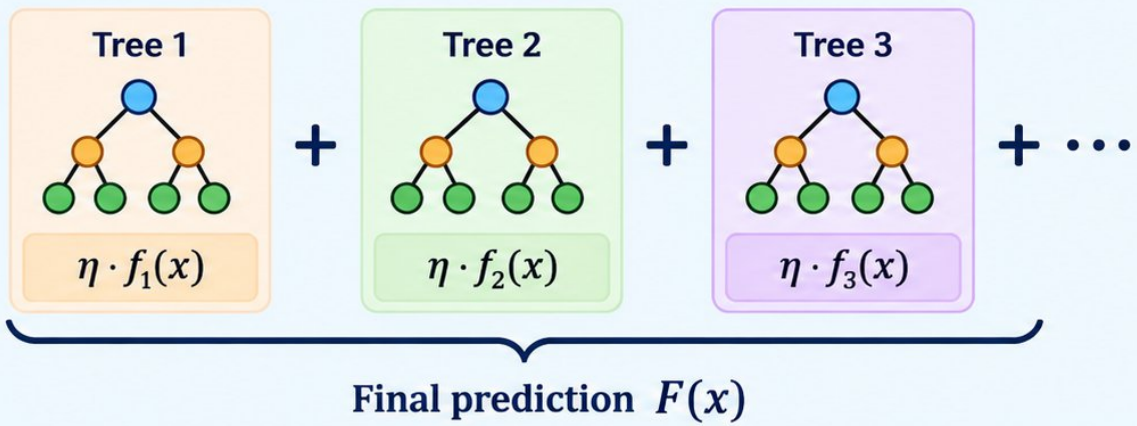
$$F_t(x) = F_{t-1}(x) + \eta h_t(x)$$



5 Final Prediction

Finally, every tree's output is combined. Each tree contributes only a small, scaled amount — and the **sum of all contributions** gives the final prediction.

That's precisely why XGBoost is called an ensemble method — the strength comes from combining many weak learners, not from any single tree.



Prediction Formula

$$F(x) = \sum_i \eta \cdot f_i(x)$$

$f_i(x)$	Prediction of tree i (output of the i-th tree)
η	Learning rate (controls the contribution of each tree)
$F(x)$	Combined ensemble output (final prediction)

Numerical example

The Dataset & Initial Prediction

Round 1

Four students, hours studied vs. marks scored. We'll boost a model to predict Marks from Hours.

Student	Hours (x)	Marks (y)
A	2	40
B	4	60
C	6	70
D	8	90

Step 1 — Initial Prediction

XGBoost starts every model with a flat baseline — typically the mean of the target values:

$$F_0(x) = \text{mean}(y) = (40 + 60 + 70 + 90)/4 = 65$$

Every student starts with the same predicted score: 65. The trees we build next will correct this baseline.

Loss, Gradient & Hessian

Round 1

We measure error with Mean Squared Error, then differentiate it to get the gradient and Hessian XGBoost needs.

1 **Loss function**

$$L = (\hat{y} - y)^2$$

Squared error between prediction $\hat{y}$ and actual y

2 **Gradient (1st derivative)**

$$g_i = \frac{\partial L}{\partial \hat{y}} = 2(\hat{y} - y)$$

How steeply the loss changes as $\hat{y}$ moves

3 **Hessian (2nd derivative)**

$$h_i = \frac{\partial^2 L}{\partial \hat{y}^2} = 2$$

Curvature of the loss — constant for MSE

Gradients & Hessians per Student

Round 1

With $F_0(x) = 65$ for everyone, we compute each student's gradient and Hessian:

Student	y	$\hat{y}$	$g_i = 2(\hat{y} - y)$	h_i
A	40	65	$2(65 - 40) = 50$	2
B	60	65	$2(65 - 60) = 10$	2
C	70	65	$2(65 - 70) = -10$	2
D	90	65	$2(65 - 90) = -50$	2

Reading the signs: students A and B were under-predicted (positive gradient pulls the prediction up), while C and D were over-predicted (negative gradient pulls it down).

The Role of λ (Regularization)

Round 1

λ is an L2 regularization parameter that controls how large leaf values are allowed to grow.

Bigger λ → smaller, more conservative leaf weights.

Smaller λ → leaves are free to fit the data closely.

λ value	Effect
0	No control → large weights
Small (1)	Mild control
Large (10, 100)	Strong control → smaller weights

Scoring the Split: Gain = 720

Round 1

Try splitting on Hours ≤ 4. Left node = {A, B}, right node = {C, D}. Using λ = 1:

Left (A, B)

$G_L = 50 + 10 = 60$

$H_L = 2 + 2 = 4$

$60^2/(4 + 1) = 720$

Right (C, D)

$G_R = -10 - 50 = -60$

$H_R = 2 + 2 = 4$

$(-60)^2/(4 + 1) = 720$

Parent (all)

$G = 0$

$H = 8$

$0^2/(8 + 1) = 0$

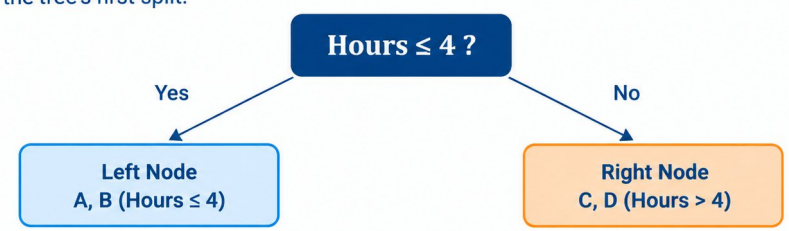
Final Gain $\text{Gain} = \frac{1}{2}\left[G_L^2/(H_L+\lambda) + G_R^2/(H_R+\lambda) - G^2/(H+\lambda) \right]$

$= \frac{1}{2}(720 + 720 - 0) = 720$ — a high gain, so this is a strong split.

The Resulting Split

Round 1

Comparing this candidate against other thresholds (≤ 2, ≤ 6) confirms Hours ≤ 4 gives the highest gain, so it becomes the tree's first split.



$w = -G / (H + \lambda)$

The optimal weight for any leaf, given the gradients (G) and Hessians (H) of the points that land in it. We'll use this next to turn each node into an actual prediction update.

Leaf Weights & the Tree 1 Update

Round 1

Region	Leaf Value (w)	Learning Rate (η)	Update
Left (A, B)	−12	0.1	−1.2
Right (C, D)	+12	0.1	+1.2

$w = -G/(H + \lambda)$: *Left* = −60/5 = −12 *Right* = 60/5 = +12

Student	Old	Update	New
A	65	−1.2	63.8
B	65	−1.2	63.8
C	65	+1.2	66.2
D	65	+1.2	66.2

Each leaf's contribution = η × w. After Tree 1, predictions move from a flat 65 toward the true pattern — the model is already learning.

Tree 2: New Residuals & Update

Round 2

With updated predictions from Tree 1, we recompute gradients — the same split structure applies, on smaller errors.

Student	y	$\hat{y}$	$g_i = 2(\hat{y} - y)$
A	40	63.8	47.6
B	60	63.8	7.6
C	70	66.2	−7.6
D	90	66.2	−47.6

Student	Old (F_1)	Update	New (F_2)
A	63.8	−1.104	62.696
B	63.8	−1.104	62.696
C	66.2	+1.104	67.304
D	66.2	+1.104	67.304

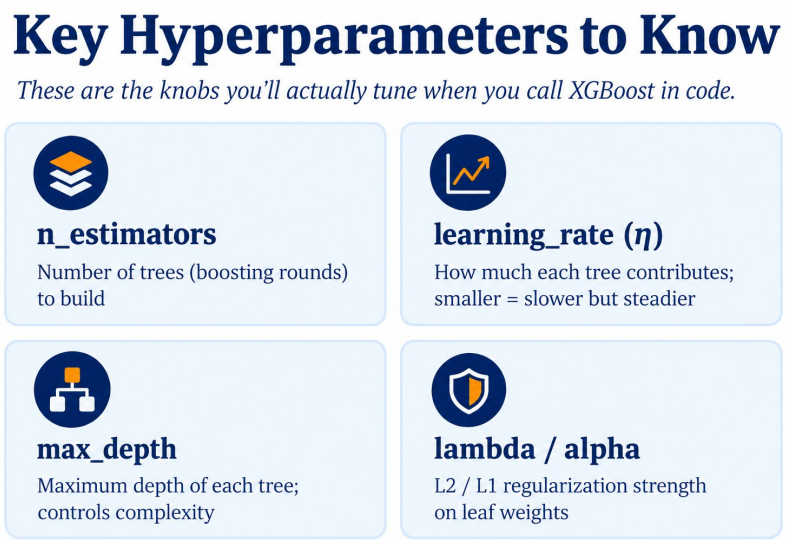
New leaf weights: $G_{left} = 47.6 + 7.6 = 55.2$, $w_{left} = -55.2/5 = -11.04 \rightarrow$ update = −1.104. Symmetric for the right leaf ($+11.04 \rightarrow +1.104$). Errors are shrinking each round.

Convergence Across Trees

Rounds 1–4

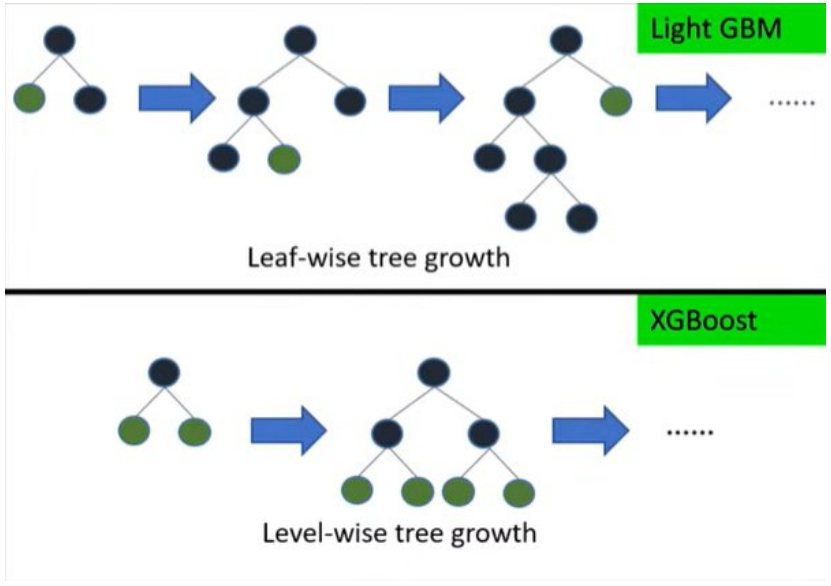
Iteration	Update Size
Tree 1	±1.2
Tree 2	±1.104
Tree 3	~±1.01
Tree 4	~±0.9

- XGBoost does not build one perfect tree. It builds many small trees, each correcting the previous errors, until the prediction becomes very close to the actual value.
- Many weak decision trees, added sequentially, each correcting the last.
- Uses both gradient and Hessian (Newton-style steps) — not plain gradient descent.
- Gain decides the best split; λ, α (alpha), and η keep the trees from overfitting.
- Parallelized, cache-aware engineering is what makes it genuinely fast at scale.
- Final prediction = the sum of every tree's small, learning-rate-scaled contribution.
- L2 regularisation(λ)- penalizes large leaf weights.
- L1 regularization(α -alpha) -encourages some leaf weights to become zero.
- Learning rate- reduces the contribution of each new tree.



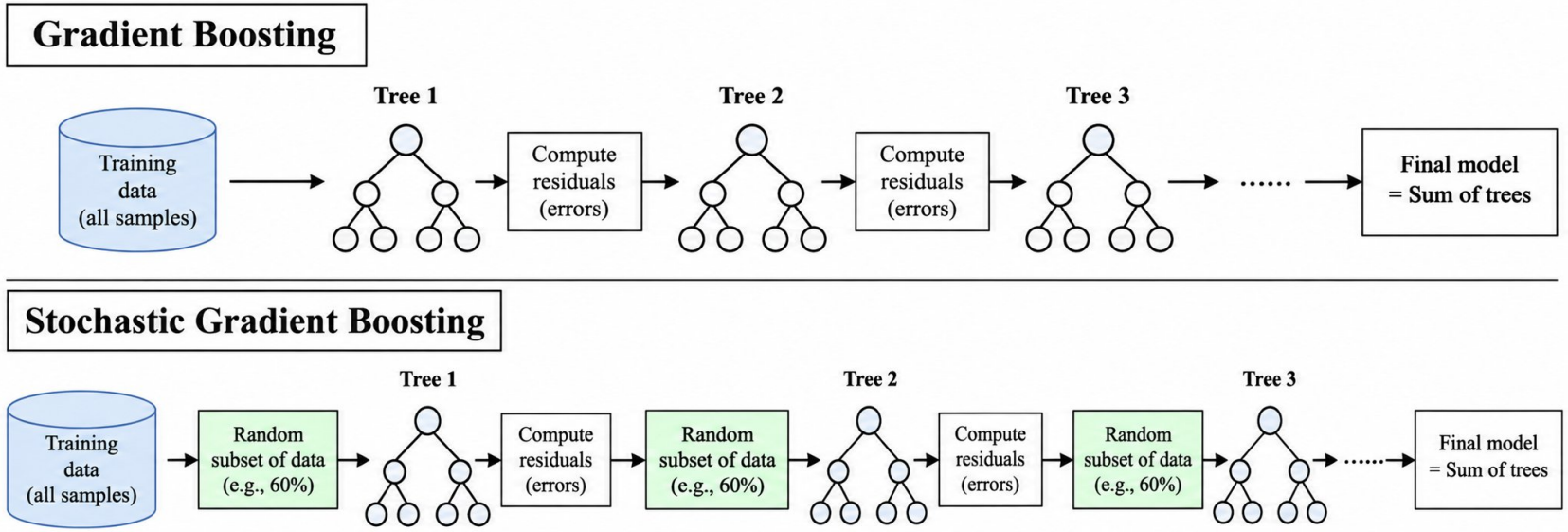
Light GBM Vs XGBoost

- Fast, distributed, high-performance gradient boosting framework based on the decision tree algorithm.
- It splits the tree leaf-wise. (Others do tree-wise or level-wise.)
- That is it selects the leaf that gives the greatest reduction in loss and splits that leaf, even if it is deeper than other leaves.
- Can lead to overfitting, which can be minimized by defining the depth for splitting. Many hyperparameters can be optimized.
- LightGBM checks which leaf should split next to reduce loss the most?
- XGBoost's level-wise approach checks which eligible nodes should split at this level?
- **accuracy increases and faster than XGBoost**
- good for image segmentation, breast cancer detection



stochastic gradient boosting : is a gradient boosting technique in which each new decision tree is trained using a randomly selected subset of the training data rather than the entire dataset.

- Like ordinary gradient boosting, each new tree tries to correct the errors made by the existing ensemble. The additional random sampling can reduce computation and help reduce overfitting.
- Gradient boosting learns sequentially by correcting previous errors. Stochastic gradient boosting adds randomness by training each new tree on only a randomly selected portion of the training data.



Feature	Ordinary Gradient Boosting	Stochastic Gradient Boosting
Data used per tree	Usually all training observations	Random subset of training observations
Computation per tree	Usually higher for large datasets	Often lower
Overfitting	Can overfit	Random sampling can help reduce overfitting
Main parameters	Number of trees, learning rate, tree depth	Number of trees, learning rate, tree depth, and subsample

Key parameter = *subsample* for stochastic for tuning

Value	Meaning
1.0	Use 100% of the observations; no row subsampling
0.8	Use 80% of the observations per boosting round
0.5	Use 50% of the observations per boosting round

The subset is generally sampled anew for each boosting iteration. A smaller value introduces more randomness, but too small a value can reduce learning quality.

Homogeneous Ensemble Learning

- An ensemble learning technique in which all the base models belong to the same type. Bagging and Boosting are examples because they use the same type of base learners with weighted inputs or contributions.

Heterogeneous Ensemble Learning

- An ensemble learning technique that combines different types of base learning models, such as SVM, ANN, and Decision Trees.
- Stacking ensemble learning is an example of this method.
  - It trains different types of models (e.g., decision tree, logistic regression, SVM) and then uses another model (meta-learner) to combine their predictions. Eg: A neural network combining predictions from decision trees and logistic regression.

Advantages of Ensemble Learning

- Improves accuracy compared to a single model.
- Reduces overfitting by averaging multiple models.
- Works well for complex problems.

Disadvantages

- Higher computation and memory requirements.
- Harder to interpret (especially when combining many models).
- Requires careful tuning of parameters.